Adaptive Load Balancer: Deliverable 2 files explained

Team 9 · Project 20 · every file in adaptive-lb-sdn, what it contains and what it does · 14 files, about 1,680 lines

1. All files at a glance

File	Lines	Folder role	Purpose
app/server.py	109	Socket programs	Threaded TCP server with a worker limit (capacity); sends UDP load reports every 0.25 s
app/client.py	70	Socket programs	Concurrent TCP load generator; one new connection per request; writes a CSV row per request
controller/balancer.py	82	SDN brain	Decision logic: static round-robin or adaptive score, draining, dead detection (no Ryu, unit-testable)
controller/lb_controller.py	221	SDN brain	Ryu app: ARP for the VIP, load reports, link monitoring, NAT flow rules, deletes a dead server's rules
topo/topo.py	46	Network	Mininet topology: s1, s2, s3; h1–h4; srv1–srv3; link A and link B at 10 Mbit/s
experiments/run.py	86	Experiments	Runs one experiment end to end (E1–E6, static or adaptive) and saves the CSVs
experiments/run_all.sh	17	Experiments	Runs all 18 runs (E1–E5 both modes, E6 at 1/2/4/8 threads both modes), then the analysis
analysis/analyze.py	131	Results	Turns the CSVs into summary.csv, summary.md and six graphs
tests/test_balancer.py	82	Tests	7 tests of the decision logic with a fake clock
tests/test_socket.py	37	Tests	Real server and client on localhost, plus a malformed-request check
dashboard/server.py	183	Website	Local web server: runs experiments and tests and serves results as JSON
dashboard/index.html	476	Website	Tabbed page with animations, Run buttons, charts and tables
README.md	139	Docs	Topology, how it works, setup, demo, experiments, tests, rubric map
.gitignore	1	Docs	Tells git to ignore __pycache__/ folders
results/	–	Output	Empty until you run experiments; run.py and analyze.py write here

2. Socket programs (app/)

File	Part	What it does
server.py	Options	--name (required), --port 8080, --workers 8 (capacity), --slow 1.0 (2.0 = half speed), --report 10.0.0.254:9999 or none, --interval 0.25 s
	main()	Opens a TCP socket on port 8080, accepts clients, gives each connection its own thread
	handle(conn)	Reads {"id", "work_ms", "size_kb"}; waits for a free worker slot; does the work (stretched by --slow); replies {"id", "server", "queue_ms", "payload"}; bad input gets {"error": "bad request"}
	do_work()	Simulates processing time for a request
	load_agent()	Background thread: every 0.25 s sends UDP {srv, active, cap, cpu, done}: active requests, capacity, CPU 0–1, and requests finished since the last report
client.py	Options	--vip 10.0.0.100, --port 8080, --threads 4, --duration 30 s, --work 100 ms, --size 4 KB, --think 0.05 s, --out (CSV file)
	request()	Opens a new TCP connection, sends the JSON request, reads the reply and times it
	worker()	Each thread keeps sending requests until --duration ends, pausing --think between them
	main()	Starts the threads and writes one CSV row per request: ts, client, latency_ms, server, ok, error

3. SDN controller (controller/)

File	Part	What it does
balancer.py	Settings	LINK_BPS 10 Mbit/s · DRAIN_AT 0.9 (stop new connections at 90% load) · RESUME_AT 0.7 (return below 70%) · DEAD_AFTER 3 s (no report = down)
	Server class	One server: name, IP, MAC, which link it is behind (A or B), and its latest load
	Balancer modes	**static**: round-robin, ignores load and failures (the baseline) · **adaptive**: lowest score among healthy servers
	report()	Saves a load report and works out the completion rate from "done"
	set_link_rate() / link_util()	Store and read how busy link A and link B are
	check_dead()	Marks servers down after 3 s without a report
	est_active()	Estimated connections = reported active + new since the report − completion rate × time since the report
	score()	**0.5 × (est. connections ÷ capacity) + 0.2 × CPU + 0.3 × link use**
	pick()	Returns the chosen server; adaptive skips draining and dead servers
lb_controller.py	Settings	VIP 10.0.0.100:8080 · reports to 10.0.0.254:9999 · LINK_PORT A = s1 port 5, B = s1 port 6 · FLOW_IDLE 10 s · a cookie number per server to tag its rules
	features()	When a switch connects: installs table-miss → controller and UDP reports → controller
	packet_in()	Sorts each packet: ARP for the VIP, load report, new connection at s1, or normal traffic
	_arp_reply()	Answers ARP for 10.0.0.100 and 10.0.0.254 (no real host owns them)
	_on_report()	Passes the server's report to Balancer.report()
	_new_flow()	On a new SYN: pick(), then installs forward and reverse NAT rules on s1 tagged with the cookie; a repeated SYN goes to the same server
	_learning_switch()	Normal switching for host ARP, iperf and direct traffic
	_monitor()	Every 1 s: requests s1 port stats, checks for dead servers, deletes a dead server's rules (adaptive mode)
	port_stats()	Turns the byte counters on ports 5 and 6 into link A and B usage
	_delete_flows()	Removes every rule with one cookie, so all of a server's rules go at once
	_write()	Logs picks, state changes, scores and link use to controller.csv

4. Network (topo/topo.py)

```
h1 h2 h3 h4 --- s1 --(port 5, link A, 10 Mbit/s)-- s2 --- srv1 (10.0.0.11), srv2 (10.0.0.12)
                \-(port 6, link B, 10 Mbit/s)-- s3 --- srv3 (10.0.0.13)
```

Part	What it does
build()	Creates 3 OpenFlow 1.3 switches connected to Ryu at 127.0.0.1:6653, 4 clients, 3 servers and the two 10 Mbit/s links; used by run.py
Run on its own	sudo python3 topo/topo.py opens the Mininet CLI for trying things by hand
--user-dp	Only for machines without the Open vSwitch kernel module (not needed on your VM)

5. Experiments (experiments/)

Scenario	What changes	Event at one third of the run
E1 Balanced	3 equal servers	None
E2 Unequal servers	srv3 has 2 workers instead of 8 and runs 2× slower	None (weak from the start)
E3 Hot server	Normal servers	h4 floods srv1 directly with slow requests, bypassing the balancer
E4 Server failure	Normal servers	The srv2 process is killed
E5 Link congestion	Normal servers	h4 sends 9 Mbit/s of iperf UDP across link A
E6 Scale-up	E1 repeated at 1, 2, 4 and 8 threads per client host	None

File	What it does
run.py	Options: --scenario E1–E6, --mode static|adaptive, --threads 4, --duration 30, --out results. Steps: start Ryu → build network → start 3 servers → start clients on h1–h3 → trigger the event → save h1/h2/h3.csv, controller.csv and meta.json to results/<scenario>_<mode>_t<threads>/
run_all.sh	Runs E1–E5 in both modes (10 runs) and E6 at 1/2/4/8 threads in both modes (8 runs): 18 runs, cleaning Mininet between runs, then runs analyze.py. DURATION sets the run length (default 30 s)

6. Analysis (analysis/analyze.py)

Part	What it does
load_runs()	Reads every run folder; calculates throughput, mean and p95 latency, error %, and each server's share
jain()	Jain's fairness index: how evenly the load was spread
bars()	Bar charts comparing static and adaptive
timeline()	Requests per second over time with the event marked (E3, E4, E5)
scale()	Latency versus thread count (E6)
Output	results/summary.csv, summary.md, latency.png, errors_fairness.png, e3_timeline.png, e4_timeline.png, e5_timeline.png, e6_scale.png

7. Tests (tests/)

File	Test	Checks that
test_balancer.py	static_is_round_robin_even_when_down	Static mode keeps taking turns, even to a dead server
	adaptive_prefers_least_loaded	Adaptive picks the lowest score
	pending_spreads_flows_between_reports	A burst of connections between reports is spread out
	drain_and_resume_hysteresis	A server stops at 90% load and returns below 70%
	dead_server_skipped	A server silent for 3 s is skipped
	fast_server_finishes_its_pending_flows	A fast server's estimate drops as it finishes work
	congested_link_shifts_to_other_path	A busy link A pushes traffic to srv3 on link B
test_socket.py	server_and_client	Real server and client on localhost: 4 threads for 2 s, more than 20 successful requests
	malformed request	Bad input gets an error reply instead of crashing the server

8. Website (dashboard/)

File	Part	What it does
server.py	Run	sudo RYU=$HOME/ryu/bin/ryu-manager python3 dashboard/server.py → http://localhost:8000 (VM only)
	GET /	Sends the website page
	GET /api/results	Reads results/ and returns each run's summary and timeline as JSON
	GET /api/status	Progress of the experiment that is running
	POST /api/run	Starts an experiment (static, then adaptive) in the background; one at a time; needs sudo
	POST /api/tests	Runs both test files and returns the output
index.html	Tabs	Overview · How it works · E1–E6 · Tests
	Experiment tab	Slowed-down animation (static vs adaptive), Run button with 20/30/60 s choice, progress bar and log, then key numbers, bar charts, timelines and a full table
	Offline	Shows built-in reference results, labelled as such, when the VM backend is not reachable

9. Documentation

File	What it contains
README.md	Topology · How it works · Setup (Mininet, OVS, Ryu) · Live demo (two terminals) · Run all experiments (E1–E6 table) · Results (run the experiments to fill) · Tests · Rubric map
.gitignore	Ignore __pycache__/ when uploading to GitHub

10. How it all works together

```
run_all.sh ──> run.py (×18) ──> starts lb_controller.py (+ balancer.py)
                              ──> builds the topo.py network
                              ──> starts server.py ×3 and client.py ×3, triggers the event, saves CSVs to results/
           ──> analyze.py ──> summary tables and graphs

dashboard/server.py + index.html ──> run any experiment from the browser and see the charts
tests/ ──> check the logic and the sockets without the network
```

In one line: servers report their load over UDP; Ryu scores each server on load, CPU and link use, sends each new TCP connection to the best healthy one with NAT rules on s1, and the experiments compare this with static round-robin.

11. Commands to run the project

Only the commands you need. Run them in a terminal in the Ubuntu VM, starting with **cd ~/adaptive-lb-sdn**. Experiments need **sudo** (Mininet requires root) and **-E** passes your RYU path through sudo.

11.1 Tests: all at once

Command	What it does	Output and what it indicates
`python3 tests/test_balancer.py`	Runs all 7 decision-logic tests	Seven "ok test_..." lines: all logic checks passed. An AssertionError names the test that failed
`python3 tests/test_socket.py`	Runs the real TCP server and client on localhost	"ok test_server_and_client": the server answered more than 20 requests and rejected a malformed request without crashing

11.2 Tests: one at a time

Pattern: `python3 -c "import sys; sys.path.insert(0,'tests'); import test_balancer as t; t.TEST_NAME(); print('ok')"` prints **ok** if that test passes, or an AssertionError if it fails.

Command	Checks that
`python3 -c "import sys; sys.path.insert(0,'tests'); import test_balancer as t; t.test_static_is_round_robin_even_when_down(); print('ok')"`	Static mode keeps taking turns, even sending to a dead server
`python3 -c "import sys; sys.path.insert(0,'tests'); import test_balancer as t; t.test_adaptive_prefers_least_loaded(); print('ok')"`	Adaptive picks the server with the lowest score
`python3 -c "import sys; sys.path.insert(0,'tests'); import test_balancer as t; t.test_pending_spreads_flows_between_reports(); print('ok')"`	A burst of new connections between reports is spread out
`python3 -c "import sys; sys.path.insert(0,'tests'); import test_balancer as t; t.test_drain_and_resume_hysteresis(); print('ok')"`	A server stops getting work at 90% load and returns below 70%
`python3 -c "import sys; sys.path.insert(0,'tests'); import test_balancer as t; t.test_dead_server_skipped(); print('ok')"`	A server silent for 3 s is marked down and skipped
`python3 -c "import sys; sys.path.insert(0,'tests'); import test_balancer as t; t.test_fast_server_finishes_its_pending_flows(); print('ok')"`	A fast server's load estimate drops as it finishes work
`python3 -c "import sys; sys.path.insert(0,'tests'); import test_balancer as t; t.test_congested_link_shifts_to_other_path(); print('ok')"`	A busy link A pushes new connections to srv3 on link B
`python3 tests/test_socket.py`	The socket test (it is the only test in its file)

11.3 Experiments: one at a time

Each command runs for 30 s and ends with **done -> .../results/<scenario>_<mode>_t<threads>**, the folder holding h1/h2/h3.csv, controller.csv and meta.json. Run the static and adaptive commands of a scenario to compare them.

Commands (static, then adaptive)	Scenario	What the results should show
`sudo -E python3 experiments/run.py --scenario E1 --mode static` `sudo -E python3 experiments/run.py --scenario E1 --mode adaptive`	Balanced: 3 equal servers	Both modes should be about the same
`sudo -E python3 experiments/run.py --scenario E2 --mode static` `sudo -E python3 experiments/run.py --scenario E2 --mode adaptive`	Unequal: srv3 has 2 workers and is 2× slower	Adaptive should give srv3 much less work: higher throughput, lower p95
`sudo -E python3 experiments/run.py --scenario E3 --mode static` `sudo -E python3 experiments/run.py --scenario E3 --mode adaptive`	Hot server: h4 floods srv1 at one third of the run	Adaptive should move new connections away from srv1: lower p95 after the event
`sudo -E python3 experiments/run.py --scenario E4 --mode static` `sudo -E python3 experiments/run.py --scenario E4 --mode adaptive`	Failure: srv2 is killed at one third of the run	Adaptive should stop errors within about 3 s; static keeps failing a third of requests
`sudo -E python3 experiments/run.py --scenario E5 --mode static` `sudo -E python3 experiments/run.py --scenario E5 --mode adaptive`	Busy link: 9 Mbit/s iperf on link A	Adaptive should send more than a third to srv3 on link B

`sudo -E python3 experiments/run.py --scenario E6 --mode static --threads 1`
`sudo -E python3 experiments/run.py --scenario E6 --mode adaptive --threads 1`
`sudo -E python3 experiments/run.py --scenario E6 --mode static --threads 2`
`sudo -E python3 experiments/run.py --scenario E6 --mode adaptive --threads 2`
`sudo -E python3 experiments/run.py --scenario E6 --mode static --threads 4`
`sudo -E python3 experiments/run.py --scenario E6 --mode adaptive --threads 4`
`sudo -E python3 experiments/run.py --scenario E6 --mode static --threads 8`
`sudo -E python3 experiments/run.py --scenario E6 --mode adaptive --threads 8`
	Scaling: E1 at 1, 2, 4 and 8 threads per client	Latency rises with threads in both modes; the curves should stay close

Command	What it does	Output and what it indicates
`... --duration 60`	Makes a run last 60 s instead of 30	Steadier numbers, slower to run
`python3 analysis/analyze.py`	Builds the summary and graphs from whatever runs you have	Prints one row per run and writes summary.csv, summary.md and the PNG graphs. "no finished runs" means run an experiment first

11.4 Experiments: all at once

Command	What it does	Output and what it indicates
`sudo -E ./experiments/run_all.sh`	Runs all 18 runs (E1–E5 both modes, E6 at 1/2/4/8 threads both modes), then analyze.py. About 15 minutes	One "done ->" line per run, then the summary table: everything is measured and the graphs are made
`cat results/summary.md`	Shows the results table	One row per run: compare static and adaptive. Adaptive should win clearly in E2, E3 and E4

11.5 Website

Command	What it does	Output and what it indicates
`sudo RYU=$HOME/ryu/bin/ryu-manager python3 dashboard/server.py`	Starts the dashboard	"Dashboard on http://localhost:8000 (Ctrl+C to stop)". Open that address in Firefox in the VM; the top-right shows "VM: leshawnlinux" when connected
`Run button on an E1–E6 tab`	Runs that experiment in static, then adaptive mode	Progress bar and log, then key numbers, bar charts, timelines and a table from your VM
`Run tests on the Tests tab`	Runs both test files	The same "ok ..." lines as 11.1

11.6 If something breaks

Command	What it does	Output and what it indicates
`sudo mn -c`	Cleans up leftover Mininet switches and links	"Cleanup complete". Use it when a run hangs or the network will not start, then run the command again
`sudo chown -R $USER results`	Makes the results folder yours again	No output. Fixes "Permission denied" from analyze.py